

## Semana 7 — Guía de la semana

# Guía de ejercicios · Semana 7 — Redes Neuronales (dígitos MNIST)

Parte de la guía oficial del curso IA Sup-rate, dividida por semana.

### A. Objetivo de la semana

Construir y entrenar la primera red neuronal y ponerla a leer números escritos a mano. Es el clásico "hola mundo" de la IA.

### B. Qué es una red neuronal (versión simple)

Un conjunto de neuronas diminutas organizadas en capas. Cada píxel de la imagen entra, cada neurona multiplica por pesos y vota, y la última capa elige el dígito más probable. Entrenar consiste en ajustar esos pesos para acertar:

```
pixeles -> [capa oculta de neuronas] -> "es un 7" (81 %)
```

Se usa `MLPClassifier` de scikit-learn para entender el concepto sin pelear con frameworks pesados. En producción se usa TensorFlow o PyTorch, pero la idea (capas, pesos y votos) es la misma.

**Nota sobre internet:** `fetch_openml("mnist_784")` descarga internet la primera vez. Si falla, el código tiene un plan B: usa `load_digits` (8x8), que ya viene instalado. Es ideal para Google Colab.

### C. Archivos de esta semana

| Archivo                       | Para qué                          |
|-------------------------------|-----------------------------------|
| <code>reto_mnist.py</code>    | El reto como script               |
| <code>reto_mnist.ipynb</code> | El mismo reto en notebook         |
| <code>autochequeo.py</code>   | Comprueba automáticamente el reto |

### D. Cómo resolver el reto

Completar los `TODO`:

| Paso | Qué hacer                                                 | ¿Dejarlo en TODO? |
|------|-----------------------------------------------------------|-------------------|
| 1–2  | Cargar MNIST (con plan B) y ver los dígitos como imágenes | Ya está           |
| 3    | Normalizar los píxeles a 0–1                              | Tú                |
| 4    | Separar train/test con <code>stratify</code>              | Tú                |

| Paso | Qué hacer                                                                        | ¿Dejarlo en TODO? |
|------|----------------------------------------------------------------------------------|-------------------|
| 5    | Definir la red ( <code>MLPClassifier</code> , capas ocultas)                     | Tú                |
| 6    | Entrenar                                                                         | Tú                |
| 7    | Evaluar: accuracy + matriz de confusión                                          | Tú                |
| 8    | Reto (opcional): probar arquitecturas (32,) , (64,) , (128, 64) , (256, 128, 64) | Opcional          |

## E. Resultados del equipo

- Mejor arquitectura: \_\_ con `max_iter=__` · **Accuracy:** \_\_\_\_ %
- Dígitos que más confundimos: \_\_ con \_\_\_\_

Experimentos:

| Arquitectura   | <code>max_iter</code> | Accuracy | Tiempo aprox. |
|----------------|-----------------------|----------|---------------|
| (32,)          | 20                    |          |               |
| (64,)          | 20                    |          |               |
| (128, 64)      | 30                    |          |               |
| (256, 128, 64) | 30                    |          |               |

Conclusión sobre qué arquitectura funcionó mejor y por qué: ...

Entregables de imágenes: 1 de dígitos de ejemplo + 1 de errores (sí / no).

## F. Autochequeo del equipo

- ☐ Se sabe cuántos valores de entrada tiene cada imagen y por qué.
- ☐ Se puede explicar qué hace una neurona ( $\text{peso} \times \text{entrada} + \text{voto}$ ).
- ☐ Se entiende para qué se normaliza.
- ☐ Se compararon al menos 3 arquitecturas y se sacaron conclusiones.
- ☐ `ENTREGA.md` se subió en la rama de entrega de la semana 7.

## G. Rúbrica (20 puntos)

| Criterio                                                  | Puntos |
|-----------------------------------------------------------|--------|
| Normalización y split con <code>stratify</code> correctos | 4      |
| Red definida y entrenada sin errores                      | 6      |
| Evaluación con accuracy + matriz de confusión             | 5      |
| Experimentación de arquitecturas con conclusión           | 5      |

## H. Reto relámpago

Pensar qué dígitos se confunden más entre sí (4/9, 3/8, 1/7) y dibujarlos lado a lado: también les cuestan a los humanos.

## I. Recursos de apoyo

---

- `tools/pistas/semana-07.md` — pistas por TODO.
- `python autochequeo.py` dentro de esta carpeta.
- `tools/06-errores-comunes.md` — errores típicos de Python.